ADTP — Phase 13

CLI Design Survey

Ashir Tech | ashir.world

Overview

Phase 13 is the CLI layer. It gives developers, server administrators, CI/CD pipelines and power users a terminal interface to every ADTP operation — file transfer, agent instruction, peer management, audit, session control and configuration — without opening the Desktop App. It is built as the `adtp-cli` crate already scaffolded in the workspace from Phase 5, now fully realised.

The CLI is designed around three principles:

- **Fast** — the common operation is always the shortest command. Smart defaults eliminate flags you would repeat on every call.
- **Effective** — every operation available in the Desktop App GUI is available here. Nothing is GUI-only.
- **Nice** — coloured output, interactive pickers, live status updates, a REPL mode and the embedded agent reachable by plain English from the terminal.

Entry Point & ASCII Art

When `adtp` is run with no arguments, or with `--help`, the ASCII art header prints first in gold (`\x1b[38;5;214m` for 256-colour terminals, `\x1b[38;2;201;168;76m` for truecolour).

The ASCII art is user-supplied and already exists. It is stored as a static string in `crates/adtp-cli/src/art.rs` and printed before every help block and before the REPL prompt.

Below the art, a one-line status bar prints immediately:

```
  v0.1.0  |  peer: a3f9bc12  |  2 peers connected  |  agent: active (claude)
```

Each segment is separated by a dim `|` separator. The peer ID is pulled live from `identity.key`. Connected peer count is pulled from the Phase 4 REST API `/api/v1/sessions`. Agent status is pulled from `/api/v1/agent/status`. If the API is unreachable, all three show as `--` in muted grey rather than erroring.

Command Structure

Every command follows the pattern:

```
adtp <verb> [noun] [target] [flags]
```

With three speed layers:

Layer	Example	When to use
Full command	`adtp send file ./report.xlsx --to work-pc`	Scripts, documentation, first use
Short alias	`adtp sf ./report.xlsx --to work-pc`	Daily terminal use
Natural language	`adtp ask "send the report to work-pc"`	Complex multi-step operations

Output Format Flags

Every command that produces output accepts these flags:

Flag	Output	Use Case
(default)	Coloured table, aligned columns, ANSI codes	Interactive terminal use
`--format json`	Raw JSON matching Phase 4 API response shape	Scripting, piping to `jq`
`--format plain`	No ANSI codes, key=value per line	Log ingestion, CI/CD
`--quiet` / `-q`	Suppress all output except errors	Silent scripting

SEND Commands

Full Commands

```
adtp send <file>
adtp send <file> --to <peer-nickname-or-peer-id>
adtp send <file> --to <peer> --wait
adtp send <file> --to <peer> --priority urgent
adtp send <file> --to <peer> --message "Here is the updated version"
```

Alias

```
adtp sf <file>
adtp sf <file> --to <peer>
```

Behaviour

- If `--to` is omitted and exactly one peer is configured as default in `cli.toml`, that peer is used silently.
- If `--to` is omitted and multiple peers exist, an interactive arrow-key picker appears listing peer nicknames. Selection confirms with Enter. Non-interactive environments (piped stdin) error with: `No default peer set — use --to <peer>`.
- `--wait` polls `GET /api/v1/transfers/{id}/status` every 500ms and renders a live progress bar (bytes sent / total bytes, percentage, elapsed time). On completion: green tick, filename, destination peer, duration. On failure: red cross, reason.
- Without `--wait`, prints the transfer ID and exits immediately with code 0.

Example Output (default format)

```
  Queued   budget.xlsx  →  work-pc   ID: a3f9bc12
  Use 'adtp status a3f9bc12' to track progress.
```

Example Output (--wait)

```
  Sending budget.xlsx to work-pc...
  ▓▓▓
```

```
adtp receive
adtp receive --pending
adtp receive --delivered
adtp receive --failed
adtp receive --from <peer>
adtp receive --limit <n>
adtp receive --session <session-id>
```

Alias

```
adtp recv
adtp recv --pending
```

Example Output

```
  ID        FILENAME         FROM         SIZE    STATUS      COMPLETED
  a3f9bc12  budget.xlsx      work-pc      1.5 MB  Delivered   2 min ago
  d7e1aa09  report.pdf       home-pc      4.2 MB  Pending     —
  f0c23b11  notes.docx       work-pc      82 KB   Failed      5 min ago
```

Status column: `Delivered` prints in green, `Pending` in gold, `Failed` in ember/red.

PEER Commands

```
adtp peer list
adtp peer add <peer-id> --name <nickname>
adtp peer remove <peer-id-or-nickname>
adtp peer trust <peer-id>
adtp peer block <peer-id>
adtp peer status
adtp peer scan
adtp peer show <peer-id-or-nickname>
```

Aliases

```
adtp pl                     # peer list
adtp pa <peer-id> -n <nick>  # peer add
```

peer list output

```
NAME         PEER ID          PATH      SINCE       STATUS
work-pc      a3f9bc12...      Direct    14 min      Connected
friend-pc    b7d2cc90...      Relay     2 hr        Connected
old-laptop   e9f1aa33...      —         3 days ago  Last seen
```

PATH column values: `Direct`, `Relay`, `QoWT`, `LAN` — coloured by type.

peer status

Shows this peer's own information:

```
Peer ID:     a3f9bc12ee7d9f230ab1...  (click or pipe to copy)
NAT Type:    Port-Restricted
Endpoint:    0.0.0.0:40001
Rendezvous:  Registered  ●  (last: 3 min ago)
Connections: 2 active
```

peer scan

Broadcasts mDNS on the local network and lists responding ADTP peers:

```
Scanning local network...

NICKNAME     PEER ID          ADDRESS          TRUSTED
office-pc    c4a1bb02...      192.168.1.42     No
dev-machine  f9e30cd1...      192.168.1.87     Yes

Use 'adtp peer add <peer-id> --name <nick>' to trust a discovered peer.
```

AGENT Commands

```
adtp agent ask "<instruction>"
adtp agent status
adtp agent activate
adtp agent deactivate
adtp agent permissions
adtp agent permissions add-path <path>
adtp agent permissions remove-path <path>
adtp agent permissions allow-command <command>
adtp agent history
adtp agent history --limit <n>
```

Top-level shortcut

```
adtp ask "<instruction>"
```

This is the fastest path to the embedded agent. Equivalent to `adtp agent ask`.

agent ask behaviour

- Sends the instruction to `POST /api/v1/agent/message`
- Streams the response as it arrives (if the provider supports streaming)
- Tool calls made by the agent are printed inline as they execute:

```
> adtp ask "find quarterly_sales.xlsx and send it to work-pc"

Agent thinking...

⟳ find_file  quarterly_sales.xlsx  →  3 results
⟳ send_file  /home/user/docs/quarterly_sales.xlsx  →  work-pc

✓ Done. Sent quarterly_sales.xlsx to work-pc (transfer ID: a3f9bc12).
```

Tool call lines print in muted purple while running (`⟳`), switch to green tick on success, red cross on failure.

agent status output

```
Agent:    Active
Provider: Anthropic
Model:    claude-sonnet-4-6
Uptime:   2 hr 14 min
Tasks:    3 completed today
```

agent permissions

```
adtp agent permissions
```

Prints the current permission state in a readable table:

```
PERMISSION         VALUE                      LAST USED
Allowed paths      /home/user/Documents       2 min ago
                   /home/user/Desktop         never
Writable paths     /home/user/adtp-received   5 min ago
Allowed commands   ffmpeg, pandoc             never
Hidden files       Denied
Remote tasks       Allowed from all peers
File size limit    10 MB
```

`adtp agent permissions add-path /home/user/Work` adds a new allowed path and writes it to `permissions.toml` immediately.

WATCH Commands

```
adtp watch <folder>
adtp watch <folder> --to <peer>
adtp watch <folder> --pattern "*.xlsx"
adtp watch <folder> --to <peer> --pattern "*.pdf" --action "summarise this"
adtp watch list
adtp watch remove <watch-id-or-folder>
adtp watch remove --all
```

watch output on registration

```
✓ Watching  /home/user/inbox  →  work-pc  (pattern: *.xlsx)
Watch ID: w_7c3e1a

Drop files into the folder to auto-send them.
Use 'adtp watch remove w_7c3e1a' to stop.
```

watch list output

```
ID        FOLDER                TO         PATTERN   SINCE
w_7c3e1a  /home/user/inbox      work-pc    *.xlsx    1 hr ago
w_2b9f04  /home/user/Desktop    friend-pc  *         23 min ago
```

AUDIT Commands

```
adtp audit log
adtp audit log --limit <n>
adtp audit log --from <date>
adtp audit log --to <date>
adtp audit log --status delivered|failed|pending
adtp audit log --peer <peer-id-or-nickname>
adtp audit verify
adtp audit export
adtp audit export --out <file.csv>
adtp audit show <transfer-id>
```

Aliases

```
adtp av                    # audit verify
adtp al                    # audit log
```

audit log output

```
  ID        FILENAME         TYPE    PEER       INTEGRITY  COMPLETED
  #L-8901   budget.xlsx      Send    work-pc    ✓          14:32:01
  #L-8900   report.pdf       Recv    friend-pc  ✓          14:28:15
  #L-8899   notes.docx       Send    work-pc    ✗ FAIL     14:15:44
```

Integrity column: green `✓` for verified, red `✗ FAIL` for failed — the most visible column.

audit verify output

```
  Verifying audit chain...

  ✓ Chain VALID — 1,204 entries verified in 0.3s


  or


  ✗ Chain TAMPERED
    First failure at entry #L-8899
    Expected hash: 7f3a...
    Got:           9c1b...
    Possible data integrity breach — do not trust entries after #L-8899
```

Exit code 0 for valid, exit code 2 for tampered — distinct from general errors (exit code 1) so scripts can handle chain failures separately.

audit export

```
adtp audit export --out audit-2026-07.csv

✓ Exported 1,204 entries to audit-2026-07.csv
```

Without `--out`, prints raw CSV to stdout — pipeable:

```
adtp audit export | grep FAIL | wc -l
```

SESSION Commands

```
adtp session list
adtp session show <session-id>
adtp session close <session-id>
adtp session transfers <session-id>
```

session list output

```
ID            PEER         STARTED      ENDED        TRANSFERS
s_9a8b7c6d    work-pc      14:10:00     Active       12
s_1f2e3d4c    friend-pc    13:55:00     14:20:00     3
```

STATUS Commands

```
adtp status
adtp status <transfer-id>
adtp status --peer <peer>
adtp status --watch
adtp status --queue
```

Alias

```
adtp st
adtp st <transfer-id>
adtp st --watch
```

adtp status (no args) output

```
┌─ ADTP Status ──────────────────────────────────────────┐
│  API          Connected      http://127.0.0.1:40003    │
│  Daemon       Connected      peer: a3f9bc12            │
│  Rendezvous   Registered     last: 2 min ago           │
│  Agent        Active         claude-sonnet-4-6         │
│  Queue        2 pending  ·  0 in progress  ·  1 failed │
└────────────────────────────────────────────────────────┘
```

adtp status --watch

Re-renders the status block every 2 seconds in-place (uses ANSI cursor control to overwrite — no scrolling spam). Ctrl+C to exit.

adtp status <transfer-id>

```
Transfer:  a3f9bc12
File:      budget.xlsx (1.5 MB)
To:        work-pc
Status:    In Progress
Progress:  ▓▓▓
```

```
api_url:              http://127.0.0.1:40003
default_peer:         work-pc (a3f9bc12...)
agent_model:          claude-sonnet-4-6
poll_interval_ms:     2000
keepalive_ms:         25000
nat_override:         (auto)
rendezvous_server:    udp://ashir.world:40004
```

adtp config set api-token

Token is written directly to the OS keychain — never stored in `cli.toml`. The terminal input is masked (no echo). Confirmation:

```
Token saved to system keychain. Not written to any file.
```

adtp config permissions

Opens the permissions configuration in an interactive terminal editor — a list of permission categories with arrow-key navigation and toggle controls, no text editor required:

```
┌─ Permissions ──────────────────────────────────────────┐
│                                                        │
│  Allowed Paths                                         │
│  ▸ /home/user/Documents                  [Remove]      │
│    /home/user/Desktop                    [Remove]      │
│    [ + Add path ]                                      │
│                                                        │
│  Allowed Commands                                      │
│    ffmpeg                                [Remove]      │
│    [ + Add command ]                                   │
│                                                        │
│  Hidden Files          [ Denied ]                      │
│  Remote Tasks          [ Allowed from all ]            │
│  File Size Limit       [ 10 MB ]                       │
│                                                        │
│  Arrow keys to navigate · Enter to edit · Q to quit    │
└────────────────────────────────────────────────────────┘
```

REPL Mode

Running `adtp` with no arguments and no subcommand drops into an interactive REPL after printing the ASCII art and status bar.

```
[ASCII ART HERE]

v0.1.0  |  peer: a3f9bc12  |  2 peers connected  |  agent: active

Type a command (without 'adtp') or ask the agent anything.
'help' for commands · 'exit' or Ctrl+D to quit.

▸ _
```

REPL behaviour

Feature	Detail
Command prefix	Commands entered without the `adtp` prefix — type `send budget.xlsx` not `adtp send budget.xlsx`
Agent access	Anything not matching a known command is passed directly to the agent — no `ask` prefix needed in REPL mode
Tab completion	Completes command names, peer nicknames, and file paths
History	Up/Down arrow cycles through command history for the session
Live output	Progress bars and `--watch` style live updates work inside the REPL
Exit	`exit`, `quit`, Ctrl+D or Ctrl+C

REPL example session

```
▸ peer list
NAME        PEER ID       PATH    STATUS
work-pc     a3f9bc12...   Direct  Connected

▸ send quarterly_report.pdf --to work-pc --wait
Sending quarterly_report.pdf to work-pc...
                          100%  4.2 MB   1.8s
✓ Delivered

▸ send the invoice to work-pc
Agent thinking...
⟳ find_file  invoice  →  2 results
? Which file did you mean?
  1. /home/user/Documents/invoice_june.pdf
  2. /home/user/Desktop/invoice_draft.pdf
▸ 1
⟳ send_file  /home/user/Documents/invoice_june.pdf  →  work-pc
✓ Done. Sent invoice_june.pdf to work-pc.

▸ exit
```

Notice the third command — a natural language instruction with no `ask` prefix — the REPL passes it to the agent directly because it does not match any known command name.

Full Command & Alias Reference

Full Command	Alias	One-line Description
`adtp send <file>`	`adtp sf`	Send a file to the default peer
`adtp send <file> --to <peer>`	`adtp sf -t`	Send to a specific peer
`adtp receive`	`adtp recv`	List incoming transfers
`adtp peer list`	`adtp pl`	List all known peers
`adtp peer add`	`adtp pa`	Add and trust a new peer
`adtp peer status`	—	Show own peer ID and NAT info
`adtp peer scan`	—	Discover peers on local network
`adtp agent ask`	`adtp ask`	Send natural language to agent
`adtp agent status`	—	Show agent active state and model
`adtp agent permissions`	—	View and edit permission rules
`adtp watch <folder>`	—	Watch folder for auto-send
`adtp watch list`	—	List active folder watches
`adtp watch remove`	—	Remove a folder watch
`adtp audit log`	`adtp al`	Show audit log entries
`adtp audit verify`	`adtp av`	Verify audit chain integrity
`adtp audit export`	—	Export audit log as CSV
`adtp audit show <id>`	—	Show one audit entry in full
`adtp session list`	—	List active and past sessions
`adtp session show <id>`	—	Detail view of one session
`adtp session close <id>`	—	Close an active session
`adtp status`	`adtp st`	Overall health and queue
`adtp status <id>`	`adtp st <id>`	Live progress for one transfer
`adtp status --watch`	`adtp st -w`	Live-updating health display
`adtp config show`	—	Print current configuration

`adtp config set <key> <value>`	—	Update a config value
`adtp config permissions`	—	Interactive permission editor

Error Output & Exit Codes

All errors print to stderr, never stdout. This means error messages do not corrupt JSON output when scripting with `--format json`.

Exit Code	Meaning
0	Success
1	General error (file not found, peer unreachable, API down)
2	Audit chain tampered — distinct code so scripts can handle separately
3	Permission denied — file or command not in allowed list
130	Interrupted by Ctrl+C

Error format:

```
Error: File not found — /home/user/budget.xlsx
Hint: Check the path and try again.
```

Errors always include a `Hint:` line with a human-readable next step — not just a code or an exception dump.

Colour System

Matches the Phase 11 Blue Eclipse palette where ANSI allows:

Element	ANSI Colour
ASCII art header	Gold `\x1b[38;2;201;168;76m`
Success ✓	Green `\x1b[38;2;63;166;107m`
Failure ✗	Ember `\x1b[38;2;200;67;10m`
In-progress ⟳	Purple-bright `\x1b[38;2;124;124;191m`
Table headers	Muted `\x1b[38;5;244m`
Peer IDs, hashes	Purple-bright `\x1b[38;2;124;124;191m`
Warnings	Gold `\x1b[38;2;201;168;76m`
Errors	Ember `\x1b[38;2;200;67;10m`
Dim separators	`\x1b[2m` (dim)

Colour is suppressed automatically when stdout is not a TTY — safe for pipes and log files without needing `--no-color`.

Crate Dependencies

Crate	Version	Purpose
clap	4, derive	Subcommand and flag parsing
reqwest	0.12, rustls-tls	HTTP calls to Phase 4 REST API
tokio	workspace	Async runtime
serde / serde_json	workspace	Deserialise API responses
comfy-table	7	Formatted table output
indicatif	0.17	Progress bars, spinners
colored	2	ANSI colour output
rustyline	14	REPL readline — history, tab completion, Ctrl+D
toml	workspace	Config file read/write
dirs	workspace	Cross-platform config path
keyring	workspace	API token from OS keychain
anyhow	workspace	Error propagation
tracing	workspace	Debug logging behind `ADTP_LOG=debug`
wiremock	0.6 dev	Mock HTTP server for tests

Files

File	Purpose
`crates/adtp-cli/src/main.rs`	Entry point — prints ASCII art, builds clap app, dispatches to subcommands
`crates/adtp-cli/src/art.rs`	Static string holding the user-supplied ASCII art
`crates/adtp-cli/src/repl.rs`	REPL mode — rustyline setup, command dispatch, agent passthrough
`crates/adtp-cli/src/commands/send.rs`	send and sf
`crates/adtp-cli/src/commands/receive.rs`	receive and recv
`crates/adtp-cli/src/commands/peer.rs`	peer list, add, remove, trust, block, status, scan
`crates/adtp-cli/src/commands/agent.rs`	agent ask, status, activate, deactivate, permissions
`crates/adtp-cli/src/commands/watch.rs`	watch, watch list, watch remove
`crates/adtp-cli/src/commands/audit.rs`	audit log, verify, export, show
`crates/adtp-cli/src/commands/session.rs`	session list, show, close
`crates/adtp-cli/src/commands/status.rs`	status, status --watch
`crates/adtp-cli/src/commands/config.rs`	config show, set, permissions
`crates/adtp-cli/src/client.rs`	HTTP client wrapper — shared by all commands
`crates/adtp-cli/src/display.rs`	Table rendering, progress bars, colour helpers, live update logic
`crates/adtp-cli/src/models.rs`	Serde structs mirroring Phase 4 API response shapes
`crates/adtp-cli/src/config.rs`	cli.toml load/save, keychain access
`crates/adtp-cli/src/errors.rs`	Error types, exit code mapping, hint generation